

CSE 341 Part 1 Test Report

BudgetLang

Student ID: 210104004817

May 2026

5.1 Overview [P1]

This report presents the Part 1 parser tests for BudgetLang. The goal of the test set is to show that the lexer and recursive-descent parser accept valid programs, construct a printable abstract syntax tree (AST), and reject malformed programs with useful error messages that include location information. This matches Sebesta's description of compiler front ends and parser responsibilities: lexical analysis produces tokens for syntax analysis, and syntax analysis must both build program structure and detect syntax errors (§4.2, §4.3.1, §4.4).

All tests were run with the command below:

```
dotnet run --project src/BudgetLang -- --ast <test-file>
```

For valid programs, the command printed an AST. For malformed programs, the command printed a parser error with a line and column number.

5.2 Valid Programs [P1]

Table ?? summarizes the three valid BudgetLang programs used for parser testing. Each program is non-trivial in the sense that it uses multiple language constructs rather than only a single statement form.

File	What the program demonstrates	Result
valid1.txt	Function declaration, foreach , if/else , transaction field assignments, array literal, function call, budget , and track .	Parsed successfully and produced a printable AST.
valid2.txt	Function declaration, transaction array construction, function call, switch , case , default , budget , and track .	Parsed successfully and produced a printable AST.
valid3.txt	Function declaration, expense aggregation, array literal, money comparison, if/else , nested transaction creation, and multiple track statements.	Parsed successfully and produced a printable AST.

Table 1: Summary of valid parser tests

Valid Program 1

This program computes total expenses from a transaction array and compares the result against a money threshold. It demonstrates that the parser handles nested block structure, field access, binary expressions, and a function call in the same source file.

```

money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;
        }
    }

    return total;
}

transaction groceries;
groceries.amount = $250;
groceries.category = "food";
groceries.kind = "expense";
groceries.note = "Weekly groceries";

transaction salary;
salary.amount = $2000;
salary.category = "job";
salary.kind = "income";
salary.note = "Part time salary";

transaction[] records = [groceries, salary];
money spent = totalExpenses(records);
budget "Food limit" $300;
track groceries;
track salary;

if (spent > $200) {
    budget "Spent" spent;
} else {
    budget "Status" $0;
}

```

Valid Program 2

This program computes total income and then uses a **switch** statement to report a monthly result. It demonstrates that the parser correctly recognizes **case** and **default** clauses as part of the switch syntax.

```

money totalIncome(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "income") {
            total = total + t.amount;
        }
    }

    return total;
}

transaction freelance;
freelance.amount = $900;
freelance.category = "sidejob";
freelance.kind = "income";
freelance.note = "Website project";

transaction bonus;

```

```

bonus.amount = $400;
bonus.category = "reward";
bonus.kind = "income";
bonus.note = "Semester bonus";

transaction[] incomes = [freelance, bonus];
money total = totalIncome(incomes);
int month = 5;

switch (month) {
    case 5:
        budget "May income" total;
    default:
        track freelance;
}

track bonus;

```

Valid Program 3

This program computes expenses, compares the total with a spending limit, and conditionally creates an alert transaction. It demonstrates that the parser handles another combination of expressions, declarations, and control flow while preserving the AST structure.

```

money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;
        }
    }

    return total;
}

transaction rent;
rent.amount = $800;
rent.category = "housing";
rent.kind = "expense";
rent.note = "Apartment rent";

transaction transport;
transport.amount = $120;
transport.category = "travel";
transport.kind = "expense";
transport.note = "Bus card";

transaction[] expenses = [rent, transport];
money total = totalExpenses(expenses);
money limit = $1000;

if (total > limit) {
    transaction alert;
    alert.amount = $10;
    alert.category = "warning";
    alert.kind = "expense";
    alert.note = "Budget exceeded";
    track alert;
} else {
    budget "Remaining budget" limit;
}

```

```

track rent;
track transport;

```

To keep the report concise, only a short AST excerpt is shown below. The full AST was printed successfully for each valid program at the terminal.

Program

Functions

Function totalExpenses : money

Parameters

items : transaction[]

This output confirms that parsing does not stop at token recognition: the parser constructs a tree-shaped representation of program structure, which is the central purpose of syntax analysis in Sebesta's presentation (§4.3.1, §4.4).

5.3 Malformed Programs [P1]

Table ?? shows five malformed programs together with the exact parser error message reported by the implementation. Each invalid test contains a targeted syntax mistake so that the error behavior is easy to explain.

File	Syntax problem	Parser output
invalid1_missing_semicolon.txt	Missing semicolon after a variable declaration.	Parsererroratline3,column1: Expected';'aftervariabledeclaration.
invalid2_missing_closing_brace.txt	Function block is not closed before the next declaration.	Parsererroratline13,column1:Unexpectedtoken'endoffile'atthestartofastatement.
invalid3_bad_if_header.txt	Missing opening parenthesis after if.	Parsererroratline3,column4: Expected'('after'if'.
invalid4_broken_field_assignment.txt	Field assignment has a dot but no field name.	Parsererroratline3,column7: Expectedfieldnameafter'.'
invalid5_missing_right_paren_call.txt	Function call is missing the closing right parenthesis.	Parsererroratline7,column36: Expected')'afterargumentlist.

Table 2: Malformed parser tests and reported errors

Malformed Program Listings

Invalid 1: Missing semicolon

```

money total = $100

budget "Limit" total;

```

Invalid 2: Missing closing brace

```

money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;

```

```

    }
}

return total;

money limit = $500;

```

Invalid 3: Bad if header

```

money spent = $200;

if spent > $100) {
    budget "Spent" spent;
}

```

Invalid 4: Broken field assignment

```

transaction rent;

rent. = $800;
track rent;

```

Invalid 5: Missing right parenthesis

```

money totalExpenses(transaction[] items) {
    money total = $0;
    return total;
}

transaction[] records = [];
money spent = totalExpenses(records;

```

These results show that the parser does not only accept correct input; it also rejects malformed input at the point where the grammar expectation fails. This behavior is consistent with Sebesta’s description that parsers must identify syntax errors while processing the token stream (§4.3.1).

5.4 Conclusion [P1]

The Part 1 parser tests support the claim that BudgetLang’s lexer and parser are functioning for the declared subset. The valid tests show that the implementation accepts non-trivial programs and produces a printable AST, while the malformed tests show that syntax errors are detected with clear diagnostic messages and source locations. This gives a usable front end for Part 1 and provides a strong base for extending the implementation with static semantics and interpretation in Part 2.